

완전동형암호 알고리즘 TFHE 스킴의 키 스위칭 하드웨어 구현

*이관표, 박지웅, 김동규

한양대학교 융합전자공학부

e-mail : *ssks501@hanyang.ac.kr*, *pjw9703@hanyang.ac.kr*, *DQKIM@hanyang.ac.kr*

Hardware Implementation of Key Switching in TFHE scheme

*Kwan - Pyo Lee, Ji - Woong Park, Dong Kyue Kim

Department of Electronic Engineering

Hanyang University

Abstract

To reduce noise, bootstrapping is essential in FHE (Fully Homomorphic Encryption). However, the secret key is changed during bootstrapping. We analyze the detailed structure of operation in Key switching which is needed to switch changed key to the original secret key. This paper proposes an implementation of Key switching hardware in TFHE.

I. 서론

산업이 발달함에 따라 데이터의 중요성이 더욱 부각되고 있다. 또한 의료, 금융과 같은 개인정보가 민감한 산업에서도 그들의 데이터를 보다 더 다양한 용도로 사용하고자 하는 움직임들이 나타나고 있다. 하지만 이런 산업에서의 데이터는 대부분 민감한 개인정보를 담고 있어 함부로 다루기 쉽지 않았다. 또한 암호화된 데이터를 연산하기 위해서는 복호화를 거치게 되는데 이 과정을 거치면서 개인정보가 유출될 위험이 굉장히 높다. 이러한 문제들을 해결하기 위한 방법으로 동형암호가 주목받고 있다.

동형암호는 암호화된 상태로 연산이 가능하고 그 연산결과가 보존되는 특징을 가지고 있다. 하지만 연산

을 함에 따라 노이즈가 생기게 되고 쌓인 노이즈가 일정 수준을 넘어가게 되면 암호화 시키려고 하는 평문까지 침범할 수 있다. 따라서 연산 횟수에 제한을 두거나 재부팅을 하는 과정을 거쳐야 하는데 이를 해결한 방법이 부트스트랩 과정이다. 이 과정을 통해 연산 횟수의 제약을 없앤 방법을 '완전동형암호'라고 한다. 부트스트랩 중 키 스위칭을 거치게 되는데 이 과정을 통해 적은 노이즈를 발생시켜 동형상 곱셈 계산 결과의 크기를 줄일 수 있다[1].

본 논문은 공개되어 있는 소프트웨어 라이브러리들 중 TFHE[2]에 대해서 다루었으며, 그 중에서도 Key switching부분의 구조를 분석하고 이를 하드웨어로 구현했다.

II. 관련이론

2.1 LWE

Learning with error'의 약자로 두 벡터의 일차결합의 모양인 'Learning without error'에 에러가 추가된 형태이다. 암호화 된 LWE는 벡터 $\vec{a}$ 와 스칼라 값 $b = \vec{a} \times \vec{s} + m + e$ 를 갖게 된다. 여기서 $\vec{s}$ 는 비밀키를 의미한다. 에러가 추가됨에 따라 기존의 Learning without error에 비해 기하급수적으로

LWE의 복잡도가 커져서 해독하기 어려워진다[3].

$$(a_0, a_1, \dots, b) = (\tilde{a}, b)$$

2.2 Lev

동일한 암호문을 서로 다른 scaling factor가 곱해진 LWE를 나열한 구조이다. 키 스위칭에서는 원래의 비밀키를 새로운 비밀키로 암호화한 형태의 Lev를 KSK_i 로 표기하며 key switching key라고 부른다. 키 스위칭 과정 중 비밀키가 유출되면 안 되기 때문에 별도의 공개키 형태인 KSK_i 가 사용된다[4].

$$\left(LWE_{s,\sigma} \left(\frac{q}{\beta^1} M \right) \times \dots \times LWE_{s,\sigma} \left(\frac{q}{\beta^l} M \right) \right) = Lev_{s,\sigma}^{\beta,l}(M) \subseteq R_q^{l \cdot (k+1)}$$

2.3 Decomposition

LWE에 크기가 큰 상수를 곱하면 노이즈가 암호문 크기에 비해 커져서 메시지를 침범하게 되는데 이를 해결하기 위해 base (β)를 이용하여 상수를 decompose한다. 이 때, decompose된 각각의 원소 A_i 는 크기가 작기 때문에 노이즈에 더 이상 큰 영향을 미치지 않는다[4].

$$A = \frac{q}{\beta} A_1 + \dots + \frac{q}{\beta^l} A_l$$

III. 본론

3.1 키 스위칭 소개

완전동형암호의 스킴 중 하나인 TFHE는 토러스(Torus) 구조를 기반으로 한다. Boolean 연산은 logic gate 수준의 연산을 처리하게 되고 연산 이후 노이즈를 줄이기 위한 부트스트래핑(bootstrapping)이 필수적이다. 부트스트래핑을 거치면 원래의 비밀키와 다른 비밀키를 가지게 되고 이 때 원래의 비밀키로 복호화하기 위해서는 키 교환(key switching)이 필요하다. 부트스트래핑의 목적은 입력과 출력에서 같은 평문과 secret key를 가지면서 노이즈만 줄이는 것이므로 Blind Rotate 과정 중 바뀐 비밀키를 원래의 비밀키로 교환해야 하는 것이다[5].

- $KeySwitch(C, KSK_i) \rightarrow \tilde{C}$
C는 부트스트래핑의 출력 값이고, KSK_i 는 원래 비밀키 S를 새로운 비밀키 S'로 암호화한 Lev형태의 암호문이다. Bootstrapping 초기에 전체 공개키인 부트스트래핑 키(BK_i)로 저장해두고 키 스위칭 모듈에서 $A_{i,j}$ 를 계

산하여 BK_i 중 KSK_i 를 불러와 모듈 내부에서 연산하면 $\tilde{C}$ 를 얻게 된다. 최종 출력 값인 $\tilde{C}$ 는 C와 동일한 M이 S'로 암호화된 LWE이다.

3.2 키 스위칭 연산

기존의 LWE에서 B를 연산하는 방법으로 B를 구하면

$$B = \sum_{i=0}^{k-1} A_i \cdot S_i = \Delta M + E$$

이 된다. 이 때, 완전동형암호이므로 각 항에 LWE 연산을 해주게 되면

$$LWE_{s'}(B) = \sum_{i=0}^{k-1} LWE_{s'}(A_i \cdot S_i) = LWE_{s'}(\Delta M)$$

이 나온다. 위 식에서 $LWE_{s'}(A_i \cdot S_i)$ 는 큰 값인 A_i 가 곱해져 있다. 이를 그대로 곱하면 노이즈가 커져서 메시지를 침범할 수 있으므로 Decomposition을 거쳐서 연산을 해 주게 된다. 그러면 아래와 같이 decomposition이 이루어진다.

$$\begin{aligned} &< Decomp^{\beta,l}(A_i), KSK_i > \\ &= A_{i,1} \cdot LWE_{s'} \left(\frac{q}{\beta^1} S_i \right) + \dots + A_{i,l} \cdot LWE_{s'} \left(\frac{q}{\beta^l} S_i \right) \\ &= LWE_{s'} \left(A_{i,1} \cdot \frac{q}{\beta^1} S_i \right) + \dots + LWE_{s'} \left(A_{i,l} \cdot \frac{q}{\beta^l} S_i \right) \\ &= LWE_{s'}(A_i \cdot S_i) \end{aligned}$$

최종적으로 키 스위칭 연산은 다음과 같다.

$$(0, \dots, 0, B) = \sum_{i=0}^{k-1} < Decomp^{\beta,l}(A_i), KSK_i > = LWE_{s'}(\Delta M)$$

IV. 구현 및 결과

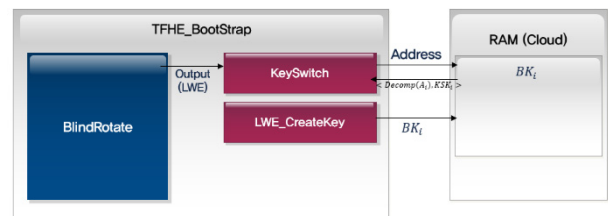


그림 1. 키 스위칭 구조도

4.1 키 스위칭 구현 방안

본 논문의 하드웨어 설계 시 파라미터는 TFHE library[2]에 사용된 set을 사용하였다. 파라미터의 의미와 값은 [표 1]에 표시하였다.

Parameter	Description	Value
q	ciphertext modulus	2^{32}
β	base of decomposition	4
l	decomposition precision level	8
n	LWE dimension	1024

표 1. LWE 파라미터

[그림 1]은 실제 구현 시 과정을 나타낸다. 부트스트랩 초기 단계에서 LWE_CreateKey에서 BK_i 를 생성하여 RAM에 저장하게 되고 20.08MB가 소모된다. BK_i 를 모두 보내기 위해 1 clock cycle에 32-bit data를 전송한다 하면 67,108,864 clock cycles이 필요하다. 키 전송을 마치고 Blind Rotate까지 완료하면 키 스위칭 모듈에 LWE를 전송하고 받은 LWE의 A_i 를 통해 $A_{i,j}$ 를 계산한다. 이때, RAM에서 $\langle Decomp^{\beta,l}(A_i), KSK_i \rangle$ 에 해당하는 주소에서 LWE를 가져와 각각의 원소들을 subtract 연산한다. 1 clock cycle에 32-bit data를 전송하고 32-bit subtractor를 1025개 두었을 때 대략 8,388,608 clock cycles이 소모된다. 각 모듈의 자세한 구조는 키 스위칭 관련 모듈 설계 챕터에서 다룬다.

4.2 키 스위칭 관련 모듈 설계

4.2.1 LWE_CreateKey

LWE_CreateKey는 현재 1024-bit key로 1024-bit random binary key(S'_i)를 encoding하고 $A_{i,j} \cdot \frac{q}{\beta^k} S'_i$ 를 LWE로 암호화하는 모듈이다. 본 논문의 모듈에서 사용된 $l(=8)$ 이므로 가능한 $A_{i,j}$ 의 값으로 0,1,2,3이 존재하므로 총 32,768($=4*8*1024$)개의 LWE를 생성한다. LWE 암호화에 필요한 키와 mask 값은 TRNG에서 생성하며 $\frac{q}{\beta^k}$ 값은 미리 $l(=8)$ 개의 32-bit 레지스터에 저장해 두고 counter_8에 따라 값을 가져오게 된다. Clock cycle에 따라 동기화되는 counter_4, counter_8, counter_1025 세 개의 카운터를 두어 BK_i 생성 시 필요한 data를 가져오게 된다.

4.2.2 KeySwitch

[그림 2]에 나타난 KeySwitch는 RAM에 저장된 BK_i 중 $\langle Decomp^{\beta,l}(A_i), KSK_i \rangle$ 에 해당하는 값을 불러와 LWE subtract 연산을 수행한다. 연산 시 RECEIVE_DATA, SUBTRACT 두 개의 state를 사용하며 counter_1025 카운터를 두어 32-bit data를 RAM에서 불러올 때 1025 clock cycles를 소모한다. LWE subtract 연산 시에는 32-bit register를 1025개 두어 1 clock cycle에 한 번에 연산하게 된다. LWE SUBTRACT 수행 동시에 다음 data를 불러오기 위해 RAM의 알맞은 주소를 계산한다. BK_i 는 RAM에 순차적으로 저장하였으므로 주소 계산은 [그림 3]와 같다.

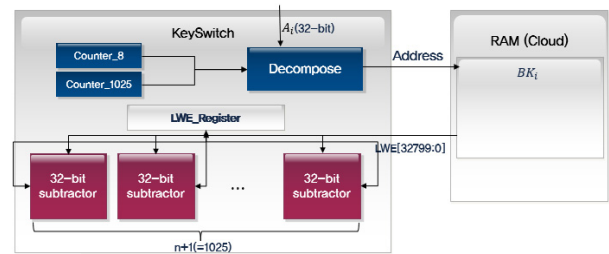


그림 2. KeySwitch 모듈 내부 구조

Algorithm 1 Decompose A_i and get address for KSK_i

Input: 32-bit constant $A_i \in \mathbb{Z}$ for $i=1, \dots, n$

Output: 32-bit RAM address which contains KSK_i

```

1: for  $i = 0$  to  $n$  do
2:    $\bar{A}_i = A_i + \text{precision offset}$ 
3:   for  $j = 0$  to  $l$  do
4:      $A_{i,j} = (\bar{A}_i \gg (32 - (j+1) * \text{basebit})) \& \text{mask}$ 
5:     if ( $A_{i,j} \neq 0$ ) then
6:       Address =  $i * h8020 + j * h1004 + A_{i,j} * h1004$ 
7:     end if
8:   end for
9: end for
10: return Address

```

그림 3. KSK_i 주소 계산 알고리즘

4.3 모듈 설계 결과

제안된 키 스위칭 하드웨어 모듈은 Verilog HDL을 사용해서 설계되었고, 삼성 28nm 공정을 통해 합성되었다. 본 논문의 설계 결과는 [표 2]와 같다.

	This Work
Max Frequency	5GHz
Area(G.E.)	515,705

CreateKey #CC	67,108,864 cycles
KeySwitch #CC	8,388,608 cycles

표 2. 키 스위칭 하드웨어 설계 결과

V. 결론 및 향후 연구 방향

본 논문에서는 토러스 구조 기반 완전동형암호 TFHE 스킴 중 키 스위칭에 대한 하드웨어 IP를 설계하였다. 상용화 시 현재 크기가 큰 공개키 BK_i 의 경량화와 파라미터에 따른 RAM과 부트스트래핑 모듈의 버스 통신 최적화가 이루어진다면 더욱 경량화된 TFHE 하드웨어의 구현이 가능할 것으로 보인다.

참고문헌

- [1] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. TFHE: Fast Fully Homomorphic Encryption over the Torus. In Journal of Cryptology, volume 33, pages 34–91 (2020).
- [2] Ilaria Chillotti. 2016. TFHE. [https://github.com/tfhe/tfhe\(2022\)](https://github.com/tfhe/tfhe(2022))
- [3] 유준수, 윤지원(2020). LWE와 완전동형암호에 대한 분석 및 동향. 정보보호학회지. 30(5). 111 - 119.
- [4] I. Chillotti. TFHE Deep Dive (2022).
- [5] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In Asiacrypt 2016 (Best Paper), pages 3 - 33.